

CustodIA

Stack técnico y justificación de cada decisión

Un runtime conversacional que convierte una intención financiera vaga en un **mandato acotado, revocable y ejecutable por máquina**. El agente investiga con datos en vivo de **The Graph**, paga por inteligencia de riesgo mediante **x402 en Hedera**, genera la interfaz mínima para que el humano fije límites, y guarda cada tarea como un subnombre **ENSv2** cuyo dueño es la wallet del usuario.

PRINCIPIO RECTOR

El agente propone. El humano autoriza el límite. El motor de políticas lo hace cumplir.

Tracks: ENS — Best Use of ENSv2 · The Graph — AI Tooling / AI Use Case · Hedera — AI & Agentic Payments

Ventana de build: 4–16 de septiembre de 2026 · **Estado al redactar:** 7–8 sept, repositorio sin commits

Documentos relacionados: [README.md](#) · [QUICKREF.md](#) (constantes verificadas) · [TEAM.md](#) · [TRACKS.md](#)

Contenido

- 1 Resumen ejecutivo y los dos filtros que decidieron todo
- 2 Arquitectura general y flujo de datos
- 3 Plataforma base (runtime, monorepo, web, API, tests, CI)
- 4 Runtime del agente (OpenAI) y UX generativa
- 5 Web3 e identidad: ENSv2 en Sepolia
- 6 Hedera: el riel de pagos x402
- 7 The Graph: inteligencia en vivo
- 8 Base de datos: por qué sí la necesitamos
- 9 Canales: web primero, Telegram como puerta de entrada
- 10 Lo que descartamos, y por qué
- 11 Verificado en vivo (no sacado de la documentación)
- 12 Riesgos aceptados y bloqueadores inmediatos
- A Anexo: versiones exactas

1. Resumen ejecutivo

Qué construimos. No es "un bot de trading con IA". Es infraestructura para operaciones financieras autónomas *seguras*: el usuario escribe en un chat lo que quiere ("mantén mi exposición ETH/USDC dentro de estos límites"), el agente investiga el mercado con datos on-chain en vivo, compra contexto de riesgo a un servicio pagado por llamada, y **genera la interfaz exacta** que el humano necesita para fijar los límites (gráfico, selector de asignación, drawdown máximo, tamaño máximo por operación, permiso de rebalanceo). Cuando la wallet firma, esas elecciones se convierten en un mandato: el agente opera dentro de él de forma autónoma, pero **no puede ampliar su propia autoridad**.

Por qué cada sponsor es estructural, no decorativo. Los jueces aplican una prueba simple: "*¿se cae el proyecto si quitamos este producto?*". La respuesta tiene que ser sí en los tres casos:

SPONSOR	ROL EN EL PRODUCTO	SI LO QUITAMOS...
ENSv2	Cada tarea persistente es un subnombre cuyo dueño es la wallet del usuario; el agente solo recibe roles acotados (EAC) sobre ese nombre.	No hay identidad de tarea, ni propiedad, ni delegación revocable. El producto no existe.
The Graph	Fuente en vivo del estado del mercado. Los <i>rangos permitidos</i> de la UI generada se derivan de la volatilidad real.	La interfaz se vuelve estática; el agente no puede razonar ni reaccionar.
Hedera (x402)	El agente paga en HBAR por el análisis de riesgo que acota el mandato.	El agente no puede poner límites informados al mandato.

Los dos filtros que decidieron el stack

FILTRO 1 — SEPARAR INTELIGENCIA DE AUTORIDAD

Cualquier pieza que mezcle "lo que el agente quiere hacer" con "lo que el agente tiene permitido hacer" fue rechazada, aunque fuera más rápida. Por eso el agente emite una *especificación tipada* de UI y nunca código; por eso la clave de Hedera vive en un proceso separado; por eso el motor de políticas es TypeScript determinista y no un LLM.

FILTRO 2 — QUEDAN 9 DÍAS, NO 12

Al redactar esto (7–8 sept) el repositorio tiene **cero commits**, y el plan original ya tenía ENSv2 funcionando el 6 de sept. Cada elección favorece lo *conocido y verificado* sobre lo elegante. El historial de commits limpio e incremental es un criterio de evaluación: hay que empezar a commitear hoy, pequeño y seguido.

2. Arquitectura general y flujo de datos

Un monorepo `pnpm` con dos aplicaciones desplegables y cinco paquetes internos. Los paquetes reflejan la división de responsabilidades de `TEAM.md`: cada persona es dueña de uno, y todos construyen contra el contrato congelado en `packages/schema` desde el día 1.

<code>packages/schema</code>	Zod: UISpec · Mandate · TaskContext · Policy ← contrato congelado día 1
<code>packages/graph</code>	adaptadores tipados de subgraphs (Uniswap V3, Aave v3, Agent0)
<code>packages/policy</code>	motor de políticas determinista: permitir / denegar + razón
<code>packages/ens</code>	ENSV2: createTask · delegateAgent · revokeAgent · resolveTask
<code>packages/agent</code>	runtime del agente (OpenAI SDK, tool-calling) → emite UISpec
<code>apps/web</code>	Next.js: chat · renderizador de UISpec · página de tarea · vista de auditoría · webhook de Telegram
<code>apps/risk-api</code>	Risk Context API con puerta x402 (Hono + Blocky402, Hedera testnet)
<code>apps/signer</code>	firmante delegado: la clave Hedera vive SOLO aquí (proceso aparte)

El flujo de punta a punta

```
graph TD
    chat["chat (web / Telegram)"] --> agente["agente investiga"]
    agente --> graph["The Graph (gateway gratuito, datos en vivo)"]
    graph --> http["llama a /risk/portfolio → HTTP 402 (precio en HBAR)"]
    http --> motor["motor de políticas: ¿gasto dentro del límite?"]
    motor -- NO --> rechazo["rechazo visible"]
    motor -- Sí --> firma["firmante delegado firma"]
    firma --> blocky["Blocky402 liquida en Hedera"]
    blocky --> json["JSON de riesgo"]
    json --> emit["agente emite UISpec (validada con Zod contra un registro de componentes permitidos)"]
    emit --> humano["el humano ajusta dentro de los rangos permitidos y FIRMA el mandato (EIP-712)"]
    humano --> acuña["se acuña el subnombre ENSv2 de la tarea (dueño = wallet del usuario)"]
    acuña --> escribe["se escribe el hash del mandato en un text record (solo el dueño puede)"]
    escribe --> delega["se delega al agente un rol acotado: solo puede escribir la clave 'status'"]
    delega --> watcher["watcher: re-consulta The Graph cada N segundos → política → acción permitida  
o cambio a NEEDS_HUMAN, o acción denegada (mostrada en auditoría)"]
```

EL MOMENTO CLAVE DE LA DEMO

Revocar el rol del agente en la tarea `a82f` → su siguiente escritura revierte con `EACUnauthorizedAccountRoles` → la tarea hermana `91ac` sigue actualizándose normalmente. Delegación acotada por recurso, en vivo, verificable en Sepolia.

3. Plataforma base

CAPA	ELECCIÓN	POR QUÉ
Runtime	Node 22 LTS · TypeScript 5.7	Aburrido a propósito. Sin sorpresas.
Monorepo	pnpm workspaces , sin Turborepo	Cinco personas necesitan fronteras de paquete que coincidan con la propiedad de <code>TEAM.md</code> . Turborepo añade configuración sin beneficio a este tamaño.
Aplicación web	Next.js 15 (App Router) · React 19	Chat, renderizador de UI generada, página de tarea, vista de auditoría y webhook de Telegram en un solo despliegue.
Risk API	Hono sobre Node, como segundo proyecto de Vercel desde el mismo repo	Los jueces pueden hacer <code>curl</code> al servicio de forma independiente; la narrativa "el agente paga a un servicio externo" se mantiene honesta. Ser Hono significa que portarlo a Railway son 20 minutos si los timeouts serverless molestan.
Validación	Zod	<code>packages/schema</code> es el contrato contra el que todos hacen stubs el día 1. Un componente desconocido o un rango fuera de política es un <i>error de validación</i> , no un fallback.
Base de datos	Neon Postgres + Drizzle (6 tablas)	Ver sección 8. La afirmación central del producto es una tarea <i>persistente</i> ; el mandato tiene que vivir fuera de la cadena porque ENS es público.
UI	Tailwind v4 + shadcn/ui	Los gráficos vienen gratis con el wrapper de Recharts de shadcn.
Lint / formato	Biome	Una herramienta, cero día de configuración de ESLint + Prettier.
Tests	Vitest + scripts <code>pnpm verify: {ens, x402, graph}</code>	Los scripts de verificación en vivo cumplen tres funciones: smoke tests nuestros, jobs de CI, y lo que el README señala a los jueces . "Clona, configura el env, corre tres comandos y mira cómo funciona" vale más que un video.
CI / CD	GitHub Actions (typecheck + test) → preview de Vercel por PR	Hace que el historial de commits parezca ingeniería, y eso se evalúa.

Los tres scripts de verificación

```
pnpm verify:ens      acuñar → delegar → escribir → REVOCAR → probar que la escritura ahora revierte  
pnpm verify:x402     402 → liquidación en Blocky402 → fila en el mirror node bajo feePayer  
0.0.7162784  
pnpm verify:graph    _meta{block{number}} en cada ID de subgraph + una consulta real
```

Estos scripts son la prueba reproducible que exige cada track ("sin datos simulados", "una petición pagada real de punta a punta"). También fallan el CI si un subgraph deja de indexar.

4. Runtime del agente (OpenAI) y UX generativa

ELECCIÓN	POR QUÉ
SDK oficial de OpenAI (<code>openai</code>), directo. <i>No</i> Vercel AI SDK ni LangGraph.	El equipo tiene tokens gratuitos en OpenAI, así que el proveedor está decidido. Vamos directo al SDK: el bucle de herramientas (<code>chat.completions.runTools</code>) y los esquemas estrictos (<code>zodFunction</code>) ya vienen incluidos, y una capa intermedia solo añadiría algo que depurar. Lo único que ganaríamos con el Vercel AI SDK es <code>useChat</code> ; un hook SSE de ~60 líneas lo cubre.
Modelo desde <code>OPENAI_MODEL</code> (por defecto <code>gpt-5.6-luna</code>)	Luna es la gama más barata de GPT-5.6 (\$0,20 / \$1,20 por millón de tokens) y está pensada justo para bucles estructurados y rígidos como el nuestro. <code>OPENAI_REASONING_EFFORT=low</code> lo mantiene rápido. Si la calidad de la justificación importa más (grabación de la demo), <code>gpt-5.6-terra</code> es un cambio de variable de entorno, no de código.
<code>zodFunction</code> + <code>runTools</code> con <code>stream: true</code>	Herramientas: <code>get_market_context</code> , <code>paid_risk_request</code> , <code>emit_ui_spec</code> . Los esquemas se convierten a JSON Schema estricto : el callback solo recibe argumentos ya validados por Zod. La comprobación de política corre al principio del callback de pago, <i>antes</i> de firmar nada. <code>maxChatCompletions: 8</code> acota el bucle.
Componentes como JSON string en <code>emit_ui_spec</code>	El modo estricto de OpenAI no admite tuplas ni referencias perezosas, que sí existen en <code>ComponentSchema</code> . Los componentes cruzan como texto JSON y <code>UISpecSchema</code> los valida al llegar: la misma frontera, distinto transporte. La plataforma añade ella misma el <code>risk_summary</code> con los números del servicio de riesgo y el recibo x402.
Prompt de sistema como mensaje <code>role: "system"</code>	Contiene el orden de los tres pasos, las formas exactas de cada componente y las dos reglas: "nunca inventes números" y "si una herramienta es denegada por política, dilo y para".
Streaming al navegador con un hook SSE de ~60 líneas	<code>runner.on("content")</code> emite los deltas de texto; los callbacks emiten los eventos de herramienta. Es un hook, no un framework.

La regla de la UX generativa

El agente **nunca emite HTML, React ni JavaScript**. Emite una especificación JSON tipada (`UISpec`) que referencia un registro de componentes permitidos. El frontend de confianza decide cómo se renderiza cada componente, qué valores se aceptan y qué advertencias son obligatorias.

```
UISpec    = { intent, components: Component[], rationale: string }
Component = price_chart | allocation_selector | range_slider | amount_selector
           | permission_toggle | risk_summary | escalation_panel

// el min/max de cada control numérico se recorta en el servidor contra el objeto de política
// mandate_signer es propiedad de la plataforma y SIEMPRE se añade – el agente no puede omitirlo
```

PATRÓN SEGURO

El agente decide **qué** contexto necesita. La plataforma controla **cómo** se presenta y **qué** puede autorizar realmente el usuario. Un nombre de componente desconocido o un rango fuera de política es un fallo de validación duro, nunca un fallback silencioso.

5. Web3 e identidad: ENSv2 en Sepolia

ELECCIÓN	POR QUÉ
viem ≥ 2.35	Las escrituras en ENSv2 necesitan ABIs hechos a mano (ver <code>QUICKREF.md</code>); <code>encodeFunctionData</code> de viem es el camino más limpio.
wagmi v2 + RainbowKit , solo Sepolia	La UI de wallet más rápida de montar y que los jueces reconocen.
siwe v3 → <code>JWT (jose)</code> en cookie <code>httpOnly</code>	Hecho a mano gana a NextAuth aquí; los jueces leen este código.
EIP-712 <code>signTypedData</code> para el mandato	Produce un hash de struct real para escribir en ENS. Un <code>personal_sign</code> no lo daría.

Modelo de nombres

<code>custodia.eth</code>	← lo registramos una vez en Sepolia (ENSv2, MockUSDC)
<code>rob.custodia.eth</code>	← subnombre ACUÑADO, dueño = wallet del USUARIO
<code>a82f.rob.custodia.eth</code>	← espacio de nombres de una tarea, dueño = usuario
<code>91ac.rob.custodia.eth</code>	← otra tarea, permisos y ciclo de vida independientes
<code>atlas.agents.custodia.eth</code>	← identidad del agente, SIN acuñar (gratis, un <code>setText</code>)

Etiquetas de tarea opacas (aleatorias) para que los registros públicos no filtren la intención financiera. Según `QUICKREF.md`, un nombre sin acuñar resuelve a través del resolver del padre, así que los agentes son gratis; solo los espacios de usuario y de tarea cuestan una escritura en el registro.

Qué va en ENS (público) y qué no

CLAVE	QUIÉN PUEDE ESCRIBIR	CONTENIDO
<code>xyz.custodia.status</code>	el agente (rol <code>ROLE_SET_TEXT</code> acotado a esta clave vía <code>authorizeTextRoles</code>)	<code>active</code> / <code>needs-human</code> / <code>completed</code>
<code>xyz.custodia.mandate</code>	solo el dueño	keccak256 del mandato firmado — <i>nunca</i> los límites en sí
<code>agent-registration[...]</code>	el dueño	Atestación ENSIP-25 hacia la entrada ERC-8004

REGLA DE PRIVACIDAD

Los registros ENS son públicos. Nunca guardar saldos, límites de pérdida detallados, secretos ni estrategia en texto plano. Solo hashes, punteros y estado grueso. Todo lo privado va detrás del backend autenticado con wallet (sección 8).

TRAMPAS CONOCIDAS (QUICKREF #5)

`authorize*` recibe un nombre **DNS-encoded**; los setters reciben un **namehash**. Mezclarlos produce reverts confusos. Los token IDs son mutables; nunca usar `setApprovalForAll` (delegación total); el bloqueo es irreversible.

6. Hedera: el riel de pagos x402

ELECCIÓN	POR QUÉ
@x402/core + @x402/hedera + @x402/hono , todos fijados en 2.25.0 exacto	@x402/hedera existe y es oficial de x402-foundation (habíamos asumido que tendríamos que firmar la TransferTransaction a mano — no hace falta). Fija @x402/core ~2.25.0 , así que todo el conjunto @x402/* debe moverse en bloque. Respaldo conocido: 2.16.0 (la versión del workshop oficial).
@hiero-ledger/sdk ^2.87	No @hashgraph/sdk : el renombre es la trampa #7 y hiero va 6 versiones menores por delante. Todo snippet anterior a 2026 está mal en algo.
Facilitador Blocky402 testnet api.testnet.blocky402.com , sin API key	Requisito duro del track. Verificado en vivo: x402Version 2 , red hedera:testnet , feePayer 0.0.7162784 . El starter oficial de Hedera apunta a x402.org/facilitator y falla el track (trampa #1).
Firmante delegado en subproceso (patrón del workshop)	La clave privada de Hedera nunca entra en la ventana de contexto del LLM . Con un bucle de herramientas del LLM leyendo datos de subgraphs, una inyección de prompt en el mismo proceso = clave robada. Una frontera de proceso (stdin → firma → stdout) lo hace estructuralmente imposible. Y es una frase de 30 segundos para un juez que lee código.
Precio en HBAR (asset 0.0.0 , tinybars), no USDC	Evita por completo la asociación de tokens HTS.
Campos numéricos estructurados; el LLM solo escribe la explicación	El motor de políticas y la UI consumen los números. Un LLM nunca decide una cifra.
Topic HCS de auditoría stretch con submitKey , sin custom fees	\$0.0008 por mensaje. Sin submitKey , cualquiera puede escribir en nuestro "registro de auditoría". Con custom fees, crear el topic cuesta \$2.00.

El protocolo x402 v2 en el cable: tres cabeceras, no una

GET /risk/portfolio	→ 402 + payment-required (la etiqueta de precio, JSON en base64)
GET /risk/portfolio + payment-signature	→ 200 + payment-response ({success, transaction, payer, network})

Las tres son JSON en base64. Un **segundo 402 se explica a sí mismo** en su cabecera payment-required : hay que decodificarla, o una caída del facilitador aparece como un bucle de reintentos silencioso. Los tutoriales v1 usan X-PAYMENT ; eso falla en silencio (trampa #2).

Cliente — API exacta, extraída del firmante de referencia

```
import { createClientHederaSigner, PrivateKey } from "@x402/hedera";
import { ExactHederaScheme } from "@x402/hedera/exact/client";
import { x402Client } from "@x402/core/client";
import { decodePaymentRequiredHeader, encodePaymentSignatureHeader } from "@x402/core/http";

const signer = createClientHederaSigner(
  accountId, // 0.0.xxxxx
  PrivateKey.fromStringECDSA(key), // claves del Portal: ECDSA; ED25519 → fromStringED25519
  { network: "hedera:testnet" },
);
const client = new x402Client().register("hedera:", new ExactHederaScheme(signer));
const payload = await client.createPaymentPayload(decodePaymentRequiredHeader(challenge));
```

El SDK construye la `TransferTransaction`, la firma y **deja vacío el slot del fee payer**: un cheque firmado sin franqueo. El facilitador se añade como pagador de la tarifa y lo envía, pero no puede cambiar ni el monto ni el destino porque nuestra firma ya los fijó. Por eso el agente casi no necesita HBAR.

TRAMPAS DE HEDERA QUE DISEÑAMOS ALREDEDOR

- **El mirror node tarda 5–6 s** en reflejar una escritura. Esperar 6000 ms antes de consultar, o se concluye que un pago exitoso falló.
- **Sin mempool; los huecos de nonce se rechazan.** Nunca `Promise.all` con transacciones del mismo firmante. Nuestro agente lanza llamadas concurrentes por defecto: la herramienta de Hedera debe serializar.
- **Faucet: portal.hedera.com con email.** El faucet anónimo crea una cuenta "hueca" que recibe pero *no puede enviar* (trampa #6). Cuesta una tarde entera.
- El límite de gasto es política del agente, no del firmante. El firmante firma lo que le pasan; `packages/policy` decide *si se le pasa algo*. Un trabajo cada uno.

7. The Graph: inteligencia en vivo

ELECCIÓN	POR QUÉ
graphql-request + respuestas validadas con Zod, ~80 líneas	No @graphprotocol/client-x402: la URL de testnet de la documentación oficial no tiene DNS (trampa #3, confirmada hoy), y el cliente valida el campo <code>chain</code> pero nunca lo aplica, así que <code>base-sepolia</code> gasta USDC real de mainnet (trampa #4). No graphclient: sus exports ESM están rotos. Zod en la frontera significa que un cambio de esquema es un error de parseo, no un <code>undefined</code> dentro de un cálculo de riesgo.
Gateway gratuito con Studio key, 100K consultas/mes	Es un proveedor en vivo que califica para el track. Sin dinero real en el camino crítico. El gateway x402 en Base está vivo (devolvió un 402 limpio hoy) pero queda como escena bonus opcional.
IDs de subgraph hardcodedos	La búsqueda del MCP es por subcadena en el nombre visible, no semántica: "DEX on Base" no devuelve nada. Uniswap V3 ETH (primario), Aave v3 ETH (plantilla 2), Agent0 Sepolia (descubrimiento).
verify:graph comprueba <code>_meta{block{number}}</code> en cada ID	El repo de Messari lleva 17 meses sin actualizar; Balancer v2 ya devuelve <code>indexing_error</code> . Ese es el fallo contra el que nos protegemos.
Caché en Postgres, escrita el día 1	No hay batching de pagos (issue #1031 abierto) y el watcher consulta en bucle. Es arquitectura, no optimización posterior.
Uniswap V3 para la tarea primaria, no Aave	Los precios se mueven cada bloque, así que "los datos en vivo provocaron una reacción" es reproducible a demanda mientras grabamos. Un health factor puede quedarse quieto 4 minutos.

Subgraphs en uso

PROPÓSITO	SUBGRAPH	ID
Precio / liquidez / swaps en vivo	Uniswap V3 Ethereum	4cKy6QQMc5tpfdx8yxfYeb9TLZmgLQe44ddW1G7NwKA6
Colateral / health factor (plantilla 2)	Aave v3 Ethereum	JCNwRypm7FYwV8fx5HhZZPSFaMxgkPuw4TnR3Gpi81zk
Descubrimiento de agentes (ERC-8004)	Agent0 Sepolia	6wQRC7geo9XYAhckfmfo8kbMRLeWU8KQd3XsJqFKmZLT

Cómo hacemos que The Graph sea estructural y no decorativo

El track premia explícitamente *razonar* sobre los datos, no mostrarlos. Así que el camino de datos es:

```
subgraph en vivo → variables de riesgo derivadas (volatilidad realizada, profundidad al notional, concentración)
                    → qué componentes de UI pide el agente + sus rangos min/max
                    → el mandato que firma el humano
                    → re-consulta en el watcher → cambio de estado o rechazo
```

LA PRUEBA DE QUE ES ESTRUCTURAL

La interfaz generada es **distinta para un par volátil que para uno tranquilo**, porque el rango permitido del selector de drawdown se deriva de la volatilidad realizada en vivo. Son 15 segundos de demo y no se puede fingir.

8. Base de datos: por qué sí la necesitamos

La razón no es el volumen de datos, sino que **la afirmación central del producto es una tarea persistente**. "Reabrir la misma tarea desde otro canal" y la vista de auditoría son, literalmente, funciones de almacenamiento. Sin persistencia, la tesis es un truco de demo.

DATO	POR QUÉ NO PUEDE VIVIR EN OTRO SITIO
El mandato firmado	ENS solo guarda el hash keccak. Los límites reales son privados (ENS es público). Si lo perdemos no podemos verificar el hash <i>ni</i> aplicar la política.
Historial de conversación	La escena de reapertura cross-canal es "búscalo por identidad de tarea".
Recibos x402 + entradas de The Graph	La vista de auditoría es un entregable evaluado.
Gasto acumulado	El momento del rechazo necesita un total corriente.
Caché de consultas a The Graph	Sin batching de pagos y con un watcher en bucle.

Por qué no las alternativas obvias

"ENS + HCS es nuestra base de datos" — la tentación web3-nativa; alguien del equipo lo propondrá. No funciona: los registros ENS son públicos, así que límites y saldos son exactamente lo que tenemos prohibido poner ahí; y HCS es solo-append con 5–6 s de retraso en el mirror: sirve como sumidero de auditoría, es inutilizable como almacén de lectura.

Memoria / archivo JSON — muere en Vercel: sistema de archivos efímero, memoria por instancia, arranques en frío. El fallo concreto: el estado desaparece a mitad de la toma de 4 minutos.

El esquema es pequeño: 6 tablas

users	wallet, ens_label, created_at
tasks	id (etiqueta opaca), user_id, ens_name, template, status, created_at, revoked_at
mandates	task_id, version, typed_data jsonb, signature, hash, signed_at ← SOLO-APPEND
messages	task_id, channel, role, content jsonb, created_at
receipts	task_id, kind (x402 hcs ens-tx), tx_id, amount, network, payload jsonb
market_cache	key, payload jsonb, fetched_at, ttl_s

DOS DECISIONES DELIBERADAS

mandates es solo-append, nunca se actualiza. Cada fila lleva su propia firma. Eso hace que "el agente no puede ampliar su propia autoridad" sea *demostrable* y no una afirmación: cada cambio de autoridad es una fila separada con una firma humana, y la auditoría renderiza la cadena. Un UPDATE destruiría esa propiedad.

El gasto acumulado se deriva como suma sobre `receipts`, no como contador mutable. Un contador puede desviarse de la realidad; una suma no, y se reconcilia contra el mirror node.

Elección: Neon Postgres + Drizzle, aprovisionado desde el marketplace de Vercel. Un clic, driver serverless, y crea una rama por preview deploy, así cada PR tiene datos aislados. No Supabase: su valor es auth y storage, y nosotros hacemos SIWE con cookie JWT. ~2 horas incluyendo migraciones, y es lo

menos arriesgado del build: hacerlo pronto precisamente *porque* es aburrido, para que todo lo demás desarrolle contra persistencia real y no contra stubs que mienten.

9. Canales: web primero, Telegram como puerta de entrada

ELECCIÓN	POR QUÉ
La app web es donde se renderiza la UI y se firman los mandatos	Nos desviamos del reporte, que proponía Telegram primero. Telegram no puede renderizar un selector de asignación ni firmar una transacción de wallet: apostar el diferenciador del producto al peor renderizador era el default equivocado.
Telegram vía <code>grammy</code> , con deep link a <code>/task/<id></code>	Mantiene la tesis de "tarea independiente del canal": el bot recibe la intención, responde con un enlace, la web renderiza y firma. Medio día de trabajo.
Slack, WhatsApp	descartados No valen un día de los 9.

La idea de fondo del reporte se conserva intacta: la plataforma de mensajería es solo transporte. La tarea pertenece a la identidad wallet/ENS, no a Telegram.

10. Lo que descartamos, y por qué

DESCARTADO	RAZÓN
Vercel AI SDK / LangGraph	El SDK de OpenAI ya trae el bucle de herramientas y los esquemas estrictos; una capa más solo añade algo que depurar. <code>useChat</code> no justifica la dependencia (sección 4).
<code>@graphprotocol/client-x402</code> , <code>graphclient</code>	Tres minas documentadas; la fuga de dinero real es descalificante por sí sola.
<code>@hashgraph/sdk</code> , <code>x402.org/facilitator</code>	Falla el requisito duro del facilitador del track (trampa #1). El SDK está renombrado (trampa #7).
Supabase	Cargaríamos auth + storage para el beneficio de una tabla; SIWE ya cubre auth.
"ENS + HCS como base de datos"	ENS es público (límites y saldos están prohibidos ahí); HCS es solo-append con 6 s de retraso.
Ejecución DeFi real, permisos de wallet ilimitados, frontend generado por el LLM	Fuera del alcance de la evaluación (reporte §11.3), y lo último es el anti-patrón que el producto existe para evitar.
Turborepo, NextAuth, Prisma	Coste de configuración sin retorno a este tamaño.
Firmar transacciones de Hedera a mano	Innecesario: <code>@x402/hedera</code> es oficial y lo hace bien.

11. Verificado en vivo (no sacado de la documentación)

Todo lo siguiente se comprobó contra infraestructura real el 7-8 de septiembre de 2026. Si un tutorial o documento lo contradice, el tutorial está mal.

COMPROBACIÓN	RESULTADO
Blocky402 testnet /health	OK {"status":"ok","version":"1.0.0"}
Blocky402 testnet /supported	OK anuncia x402Version: 2, scheme exact, red hedera:testnet, feePayer: 0.0.7162784 — coincide exactamente con QUICKREF.md
testnet.gateway.thegraph.com	SIN DNS Trampa #3 confirmada; sigue apareciendo en la documentación oficial
gateway.thegraph.com/api/x402/...	VIVO devuelve 402 sin autenticación
gateway.thegraph.com/api/subgraphs/id/...	VIVO auth error: missing authorization header → necesita Studio key
@x402/hedera	EXISTE v2.25.0, oficial de x402-foundation, exporta exact/client, exact/server, exact/facilitator; depende de @hierio-ledger/sdk 2.85.0
@hierio-ledger/sdk vs @hashgraph/sdk	2.87.0 vs 2.81.0 — hiero va por delante; el renombre es real
Workshop oficial blockydevs/wad2026-x402-workshop	Existe, último push 10 jul 2026. Usa Hono + @x402/hono + @x402/hedera en el servidor y un firmante en subproceso. De ahí sale la API exacta de la sección 6.

12. Riesgos aceptados y bloqueadores inmediatos

Riesgos aceptados

RIESGO	MITIGACIÓN
ENSv2 en beta + ABIs a mano	El ítem de mayor riesgo del build. Se ataca el día 1 después del scaffolding , no el día 6. Probar acuñar → delegar → revocar antes de tocar el frontend.
Vercel serverless + 6 s de retraso del mirror	<code>maxDuration = 60</code> en la ruta de liquidación. Hono hace que la salida a Railway cueste 20 minutos.
Deriva de versiones en <code>@x402/*</code>	Fijadas exactas en 2.25.0; respaldo conocido 2.16.0.
Un subgraph deja de indexar	<code>verify:graph</code> lo detecta en CI; segunda fuente en vivo como fallback, nunca datos simulados.
Demasiado alcance de "trading"	Acción simulada controlada por política; nunca fondos reales. No construimos un exchange.

Bloqueadores: nadie los ha empezado y frenan trabajo real

HACER HOY, ANTES DEL SCAFFOLDING — ~10 MINUTOS CADA UNO

- **API key de The Graph Studio.** El único bloqueador de toda la capa de The Graph; sin ella todo son stubs.
- **Cuenta de Hedera testnet vía `portal.hedera.com` con email.** No el faucet anónimo: crea una cuenta hueca que no puede enviar (trampa #6).
- **Registrar `custodia.eth` en Sepolia** (MockUSDC → approve → commit → esperar 60 s → register). Gratis en la práctica.
- **Primer commit.** El historial vacío es en sí mismo un riesgo de evaluación.

Qué recortaríamos sin dudar si nos retrasamos

Recibos en HCS (barato pero no exigido), la segunda plantilla de tarea (colateral en Aave), Telegram. **Nunca** se recorta: la demo de revocación, la demo de acción denegada, ni los datos en vivo.

Calendario (9 días)

FECHA	HITO
7-8 sept	Commit de la documentación, scaffolding del workspace, congelar esquemas Zod, stubs de todos los paquetes, corrida falsa de punta a punta.
8 sept	Spike ENSv2 primero: acuñar → delegar → revocar .
8-9 sept	Petición pagada x402 reproducible, de forma independiente.
9-10 sept	Adaptadores de The Graph en vivo.
10-12 sept	Runtime del agente + renderizador de UISpec + motor de políticas.
12-13 sept	Watcher / reacción autónoma, vista de auditoría, adaptador de Telegram.
14-15 sept	Endurecer, README con archivo y línea exactos por sponsor, ensayo.
16 sept	Tres cortes de video (uno por sponsor), envío.

Anexo A. Versiones exactas

Verificadas contra el registro de npm el 8 de septiembre de 2026. Fijar exacto donde se indica.

PAQUETE	VERSIÓN	NOTAS
openai	^7.12.1	chat.completions.runTools + zodFunction; modelo gpt-5.6-luna por defecto (OPENAI_MODEL)
@x402/core	2.25.0 exacto	Todo @x402/* en bloque
@x402/hedera	2.25.0 exacto	Fija @x402/core ~2.25.0
@x402/hono	2.25.0 exacto	Middleware del servidor
@hiero-ledger/sdk	^2.87.0	No @hashgraph/sdk
viem	^2.56 (mín. 2.35)	ENSv2 con ABIs a mano
graphql-request	^7.4.0	+ Zod en la frontera
hono	^4.13	Risk API
next / react	15 / 19	App Router
wagmi / RainbowKit	v2 / actual	Solo Sepolia
siwe / jose	v3 / actual	Auth con wallet
drizzle-orm + Neon	actual	6 tablas
grammy	actual	Adaptador de Telegram
zod	v4	@x402/core trae su propio Zod 3 interno; es dependencia, no peer, sin conflicto
vitest · @biomejs/biome	actual	Tests · lint/formato

Constantes de infraestructura (de QUICKREF.md)

Hedera	facilitador testnet feePayer testnet HBAR asset id faucet	https://api.testnet.blocky402.com 0.0.7162784 0.0.0 portal.hedera.com	(sin API key) (debe coincidir EXACTO) (montos en tinybars) (con email, 1000 HBAR)
ENSv2	ETHRegistry UniversalResolverV2 MockUSDC ROLE_SET_TEXT	0xbdc85dd5b15d7ecb354cd7cb6f2c50b4f2c4f0e2 0x4a1817d13e9cf196f471725176355c1234b63c70 0x768f42455a2d082e23ceef7d51e5787c82d67a39 1 << 4	(Sepolia) (mint permisivo) (admin = rol << 128)
Graph	gateway gratuito Subgraph MCP (SSE)	https://gateway.thegraph.com/api/{KEY}/subgraphs/id/{id} https://subgraphs.mcp.thegraph.com/sse	

CustodIA · ETHOnline 2026 · Documento vivo: si algo aquí contradice a QUICKREF.md , gana QUICKREF.md . Si descubres algo nuevo, edita QUICKREF.md y avisa al equipo.